

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/221932254>

New Trends in Clobber Programming

Article in ICGA Journal · September 2011

DOI: 10.3233/ICG-2011-34304

CITATIONS

0

READS

1,605

4 authors:



[Wieczorek Wojciech](#)

University of Bielsko-Biała

45 PUBLICATIONS 177 CITATIONS

[SEE PROFILE](#)



[Rafał Skinderowicz](#)

University of Silesia in Katowice

30 PUBLICATIONS 350 CITATIONS

[SEE PROFILE](#)



[Jan Kozak](#)

University of Economics in Katowice

83 PUBLICATIONS 556 CITATIONS

[SEE PROFILE](#)



[Przemysław Juszczuk](#)

University of Economics in Katowice

64 PUBLICATIONS 326 CITATIONS

[SEE PROFILE](#)

NEW TRENDS IN CLOBBER PROGRAMMING

Wojciech Wieczorek¹

Rafał Skinderowicz

Jan Kozak

Przemysław Juszczuk

Institute of Computer Science, University of Silesia, Sosnowiec, Poland

ABSTRACT

Clobber is an abstract, two-player, strategy game invented in 2001 by the combinatorial game theorists Michael H. Albert, J. P. Grossman and Richard Nowakowski. In 2005 and 2006 it was one of the events in the Computer Olympiad. Despite a very simple rule set, not only is Clobber a hard game from the perspective of the theory of combinatorial games, but also in its computer implementation. This article provides a detailed exposition of four heuristic algorithms for playing the game. These algorithms were used in programs which took four leading places in a computer Clobber tournament at Silesian University.

1. INTRODUCTION

Clobber is played on a rectangular grid (usually a ten by ten board), initially filled in a chessboard pattern with black and white stones. Players Black (Left) and White (Right) take turns; on his turn, a player moves by picking up one of his stones and ‘clobbering’ an adjacent stone of the opposite colour (replacing the clobbered stone by the one that was moved). The first player unable to move loses. Clobber was introduced by its inventors in 2002 (see Albert *et al.*, 2005). In respect of computational complexity, the following results have been achieved: (a) a solitaire (one-player) version of Clobber is NP-complete (Demaine, Demaine, and Fleischer, 2004); (b) determining who wins from a Clobber position is NP-hard when played on an arbitrary graph (Albert *et al.*, 2005); (c) the complexity of two-player Clobber is open when played on a grid (Hearn and Demaine, 2009; Albert *et al.*, 2005).

In combinatorial game theory (Conway, 2000; Berlekamp, Conway, and Guy, 2003) games such as Clobber—in which either both sides have a move or neither one does—are called all-small. Some games naturally split into components as play proceeds and can be effortlessly evaluated to numbers. In that case the sub-positions are just summed up in order to get the value of the whole game. Unfortunately, every all-small game G is infinitesimal, i.e., $-x < G < x$ for all positive numbers x (Albert, Nowakowski, and Wolfe, 2007). In practice, all-small games are often quite challenging because complex forms of evaluating results abound, and it is laborious to determine who has the advantage and by how much in each local position. Clobber’s endgames also separate into independent sub-positions which can be summed up. However, the values of the sub-positions tend to become just convoluted forms. An illustration that suggests itself is given in Figure 1. The canonical form (the value) of

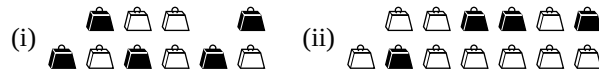


Figure 1: Two exemplary Clobber positions.

position (i) is given by

$$\{0, \{*, \{0 \mid *, \uparrow\} \mid *, \downarrow\} \parallel \downarrow*, \{0, \{\uparrow*, \{\uparrow, \uparrow* \mid 0, *\} \mid \uparrow* \mid *, \uparrow\} \mid *, \downarrow\} \parallel \downarrow*, \{0, \downarrow* \mid \downarrow*\} \mid \downarrow\},$$

for (ii) it is six times longer, while the number of symbols used to represent the value of the position (i)+(ii) is approximately 8000 times larger. This can be very inconvenient in symbolic manipulation, and makes the

¹email:wojciech.wieczorek@us.edu.pl

notation of combinatorial game theory impractical for computer play. As a result, computer programs need to incorporate more heuristic knowledge of Clobber strategy.

The aim of this paper is to acquaint other researchers with our concepts for evaluating Clobber positions and carrying on the game. We hope that it will inspire academics to improve their heuristics and to build better Clobber playing programs. Moreover, in view of the possibility of future comparisons with other methods, our programs are available to the readers (Wieczorek *et al.*, 2011). This paper is organised into three sections and two appendices. We present in Section 2 the principles that underlie our programs. They are presented in consecutive subsections in order of their performance. Concluding comments and directions for future work are contained in Section 3. Appendix A gives information on a computer Clobber tournament held on June 22, 2011, such as its venue, the names of the participants and their programs, and the playing conditions. So as to demonstrate the playing strength of the winner, *Head-scratcher*, the highlights of a selected game are given in Appendix B.

2. DESCRIPTION OF ALGORITHMS

We briefly introduce the basic terminology for Clobber. A player's representative on a board—a game piece—will be called a stone and represented in diagrams by a cubic brick as in Figure 1. Clobber positions tend to break down into independent sub-positions, each of which consists of a group of connected stones. Such a group will be called a block. If B is a block, then by $|B|$ we mean the number of stones in B provided there is at least one stone of each colour; otherwise $|B| = 0$. If $\mathcal{B}$ is a whole board then $|\mathcal{B}| = \sum_{B \in \mathcal{B}} |B|$. For a colour $c = \text{Black}$, $\bar{c} = \text{White}$, and vice versa. Let $M_c(b)$ denote the set of all possible moves for a player c in a position b , i.e., one or more blocks. Observe that $|M_c(b)| = |M_{\bar{c}}(b)|$ always holds. We will use the symbol $\mathbb{Z}$ to denote the set of integers, and $\mathbb{R}$ to denote the set of real numbers. Now we refer to the description of the algorithms mentioned in the Introduction. Notice that each of the algorithms is a heuristic, i.e., a technique which seeks good moves at a reasonable computational cost without being able to guarantee either an optimal play or a win.

2.1 Head-scratcher's algorithm

This algorithm is a hybrid of a few basic precepts, the estimation of the outcome of a sum of components, and references to game trees built for every sufficiently small position b . The sufficiently small size property implies that $|M_c(b)| \cdot |b| < P$ and $|b| < P/6$. The number P is an algorithm's parameter (in the tournament $P = 90$ was chosen). It is used to decide whether a block or a board should be evaluated: the larger is P , the larger is the increase in the level of play, but at a bigger computational cost. The pseudo-code of the algorithm is presented as Procedure FindMove. Throughout this subsection, $v(b)$ stands for the value of a position b . If a block B , for the player of colour c to play, meets all the following three conditions: (a) B is sufficiently small; (b) $v(B)$ is an integer; (c) the colour c is Black and $v(B) \geq 0$ or the colour c is White and $v(B) \leq 0$; then we say that a block B is resolved. Finally, we write $m(b)$ for the position obtained from b by making a move m .

In Procedure FindMove we can single out three parts that correspond to three stages of a play. Lines 4–14 are executed in the beginning, lines 16–19 are executed in the middlegame, whereas line 2 is executed in the endgame. The patterns from (i) to (iv) and their moves are shown in Figure 2. The most important element of the algorithm is a function v used in the evaluation of a board position. For a position b , the domain of the function is $(\mathbb{Z}_+ \times \mathbb{Z}_-) \cup \mathbb{Z}$; $v(b) \in \mathbb{Z}_+ \times \mathbb{Z}_-$ if the next player to play whether it be Black or White can force a win in b (we write this $b \in \mathcal{N}$ for short), $v(b) = 0$ if the second player to play can force a win in b (we write this $b \in \mathcal{P}$ for short), $v(b) > 0$ if Black can force a win in b regardless of who moves first (we write this $b \in \mathcal{L}$ for short), $v(b) < 0$ if White can force a win in b regardless of who moves first (we write this $b \in \mathcal{R}$ for short).² Let b be a position in which the player of colour c can force a win. Suppose that $|v(b)| = x$. Then x is the minimal number of moves played by $\bar{c}$ (with $\bar{c}$ to play first) to change the local (b 's) outcome, with c playing, or omitting his turn when required, so as to maximize $\bar{c}$'s moves. For example, $v(\text{Black stones}) = -2$ while $v(\text{White stones}) = 1$. The way in which this minimal number of moves can be determined is explained in the next paragraph. If $v(b) = (p, q)$ (i.e., $b \in \mathcal{N}$), then

$$p = \max_{\{m \in M_{\text{Black}}(b) : m(b) \notin \mathcal{N}\}} v(m(b)) + 1, \quad q = \min_{\{m \in M_{\text{White}}(b) : m(b) \notin \mathcal{N}\}} v(m(b)) - 1.$$

For example, $v(\text{Figure 1 position}) = (2, -1)$ while the value of block (i) in Figure 1 is $(1, -2)$.

²These four cases are known as outcome classes in the theory of combinatorial games.

Procedure FindMove($\mathcal{B}, c$)

```

1 if  $\mathcal{B}$  is sufficiently small then
2   return an optimal move by means of a game tree built for the board  $\mathcal{B}$ 
3 if not all blocks are sufficiently small then
4   if on the board there is a pattern of type (i) then
5     return a move that minimizes the number of remaining moves in a block of type (i)
6   else if on the board there is a pattern of type (ii) in a not resolved block then
7     return a randomly selected move of type (ii) in a not resolved block
8   else if on the board there is a pattern of type (iii) in a not resolved block then
9     return a randomly selected move of type (iii) in a not resolved block
10  else if on the board there is a pattern of type (iv) in a not resolved block then
11    return a randomly selected move of type (iv) in a not resolved block
12  else
13     $B \leftarrow \text{an element of } \text{argmax}_{B \in \mathcal{B}} (|M_c(B)| \cdot |B|)$ 
14    return a randomly selected move from  $M_c(B)$ 
15 else
16   if  $c = \text{Black}$  then
17     return  $m \in \text{argmax}_{m \in M_c(\mathcal{B})} \text{Evaluate}(m(\mathcal{B}), \bar{c})$ 
18   else
19     return  $m \in \text{argmin}_{m \in M_c(\mathcal{B})} \text{Evaluate}(m(\mathcal{B}), \bar{c})$ 

```

In this subsection we assume a game tree to be a complete game tree. An initial position is at a root and for every node there are child nodes representing Black's choices and child nodes representing White's choices. We can now identify a root's value by referring to a game tree in which each node is labelled with its outcome class. Assume that a root $r \in \mathcal{L}$ ($r \in \mathcal{R}$). Since in the position r , Black (White) can force a win, only sub-trees representing White (Black) choices for r will be processed. First, set $x(n) = 0$ for every node $n \in \mathcal{P}$; set $x(n) = 1$ for every node $n \in \mathcal{N}$; and set $x(n) = -1$ for every node $n \in \mathcal{R}$ ($n \in \mathcal{L}$). Now, let n be a node—other than the root—labelled with the class $\mathcal{L}$ (the class $\mathcal{R}$), let $\{b_i\}_{i \in I}$ be n 's child nodes representing Black's moves, and let $\{w_j\}_{j \in J}$ be n 's child nodes representing White's moves. If $\max_{i \in I} \{x(b_i)\} > \min_{j \in J} \{x(w_j)\}$ (resp. $\max_{j \in J} \{x(w_j)\} > \min_{i \in I} \{x(b_i)\}$), then $x(n) = \max_{i \in I} \{x(b_i)\}$ (resp. $x(n) = \max_{j \in J} \{x(w_j)\}$), otherwise $x(n) = \min_{j \in J} \{x(w_j)\} + 1$ (resp. $x(n) = \min_{i \in I} \{x(b_i)\} + 1$). Finally, $v(r) = \min_{j \in J} \{x(w_j)\}$ (resp. $v(r) = -\min_{i \in I} \{x(b_i)\}$), where $\{w_j\}_{j \in J}$ (resp. $\{b_i\}_{i \in I}$) are root's child nodes representing White's (Black's) choices.

The remainder of this section will be devoted to the manner in which a block's values are summed up. We will need the following assumptions: the sequence of blocks $B_1, B_2, \dots, B_s, B_{s+1}, \dots, B_{s+t}$ ($s + t \geq 1$) comprises a board position $\mathcal{B}$, where $B_i \in \mathcal{N}$ for all $1 \leq i \leq s$ and $B_{s+j} \notin \mathcal{N}$ for all $1 \leq j \leq t$; moreover, the leading blocks are arranged in non-increasing order based on the sort key $p_i - q_i$, where $(p_i, q_i) = v(B_i)$ for $1 \leq i \leq s$; the player of colour c is to play next. The addition formula is given by

$$\text{Evaluate}(\mathcal{B}, c) = \sum_{i=1}^s (v(B_i))_{i \bmod 2} + \sum_{j=1}^t (v(B_{s+j})),$$

where $(v(B_i))_0 = p_i$ and $(v(B_i))_1 = q_i$ if c is White or $(v(B_i))_0 = q_i$ and $(v(B_i))_1 = p_i$ if c is Black. An example of an evaluation can be found in Appendix B.

2.2 Rafal's algorithm

In the following description of our approach, we assume that the game is played on a ten by ten board and the cumulative computation time for selecting first and all subsequent player's moves is severely limited (see Appendix A for details). Under this assumption, a simple brute-force approach based on an exhaustive game tree search is not feasible. In the proposed algorithm, the strategy used to select the next move depends on the number of the player's stones which are adjacent to at least one of the opponent's stones. If the number is

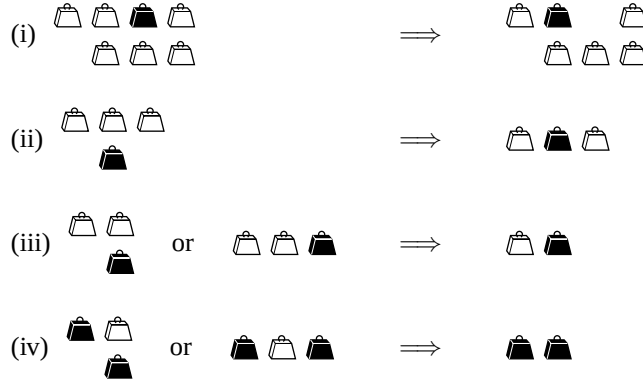


Figure 2: Four types of patterns and their recommended moves: (i) only one stone of colour c and at least two stones of a colour $\bar{c}$ forming one block $B \notin \mathcal{N}$; (ii) the possibility of playing between two stones of colour $\bar{c}$; (iii) the possibility of clobbering one of two connected stones of colour $\bar{c}$; (iv) the possibility of joining to another stone of colour c .

greater than 70, the next move is selected randomly from all available moves. Otherwise, the selection of the next move is based on the well-known minimax algorithm with *alpha-beta* pruning (Russell and Norvig, 2002). In the minimax algorithm, the game tree consisting of all possible subsequent player's and opponent's moves is explored in a depth-first manner and the best possible move is selected based on the evaluation of the game states represented by the deepest nodes. The number of nodes to be explored increases exponentially with the number of available moves, therefore the game tree can be visited only to a limited depth (level), $h_{\max}$. In our approach we used a simple heuristic to determine the value of $h_{\max}$:

$$h_{\max} = \begin{cases} \infty & \text{if } M_c(\mathcal{B}) < 15, \\ \operatorname{argmax}_{n \in \mathbb{Z}} (M_c(\mathcal{B}))^n \leq 2 \cdot 10^6 & \text{if } M_c(\mathcal{B}) \geq 15. \end{cases}$$

Of course, the condition $M_c(\mathcal{B}) < 15$ does not guarantee that the size of the game tree will be small enough for all possible board states, therefore an additional time limit of one second was set to bound the time of the game tree search. In some cases this can negatively impact the quality of selection of the next move, because only a limited portion of the game tree will be searched. The number 15 was chosen based on sample games played with a Random Player algorithm in which the next move was taken randomly from the set of all available moves.

The essential part of the minimax algorithm is its game state evaluation function, $F_c(\mathcal{B}) \in \mathbb{R} \cup \{+\infty, -\infty\}$, which indicates how good it would be for a player c to reach a given board position $\mathcal{B}$. This function is given by

$$F_c(\mathcal{B}) = \begin{cases} \infty & \text{if player } c \text{ wins,} \\ -\infty & \text{if player } \bar{c} \text{ wins,} \\ \sum_{B \in \mathcal{B}} H_c(B) & \text{otherwise,} \end{cases}$$

where $H_c(B)$ is a heuristic block evaluation function. The value of $H_c(B)$ is given by

$$H_c(B) = \begin{cases} S_c(B)/S_{\bar{c}}(B) & \text{if } S_c(B) > S_{\bar{c}}(B), \\ -S_{\bar{c}}(B)/S_c(B) & \text{if } S_c(B) \leq S_{\bar{c}}(B), \end{cases}$$

where $S_c(B)$ is a score based on the number and placement of c 's stones in the block B . The higher is the value of $S_c(B)$, the 'better' is the block B for player c . Analogously, $S_{\bar{c}}(B)$ is the score assigned to the block B based on the number and placement of $\bar{c}$'s stones.

If B contains only stones with a single colour ($|B| = 0$) then $S_c(B) = S_{\bar{c}}(B) = 0$, otherwise the score is equal to the sum of the scores for each of the player's stones in B . The score for a stone depends on the number of opponent's stones in its nearest neighbourhood, i.e., to the left, right, up, or down, and the number of own-stones placed on the diagonals, i.e., to the south-west, south-east, north-west, or north-east. This score is calculated according to the formula

$$S_c(B) = \sum_{s_c \in B} 1 + O(s_c) + D(s_c),$$

where s_c denotes a stone of colour c in B , $O(s_c)$ is the number of $\bar{c}$'s (opponent's) stones in the direct neighbourhood of a stone s_c , i.e., to the left, right, up, or down ($O(s_c) \in \{0, 1, \dots, 4\}$) and $D(s_c)$ is the number of c 's stones on the diagonals which are 'tied' with the stone. A stone is tied with its diagonal stone if both are adjacent to the same opponent's stone ($D(s_c) \in \{0, 1, \dots, 4\}$).

For example, in block (i) shown in Figure 1, the leftmost Black stone is adjacent to only one opponent's stone and it has only one (tied) diagonal stone (on the north-east). The total score for all white stones in the block (let us denote the block by B) is equal to

$$S_{\text{White}}(B) = (1 + 2 + 2) + (1 + 0 + 0) + (1 + 3 + 1) + (1 + 2 + 1) + (1 + 2 + 0) = 18,$$

analogously, the score for Black is equal to

$$S_{\text{Black}}(B) = (1 + 2 + 1) + (1 + 1 + 1) + (1 + 1 + 1) + (1 + 3 + 1) + (1 + 2 + 1) = 19,$$

and, finally, the heuristic block evaluation function value for White is equal to

$$H_{\text{White}}(B) = -S_{\text{Black}}(B)/S_{\text{White}}(B) = -19/18.$$

The key point of our evaluation function is to assign greater values to the blocks with greater ratios of the number of possible moves to the number of own-stones. Intuitively, the more possible moves for a block with few own-stones, the greater the chance to force an opponent to make the last but one or, if necessary, the last move in the block. Despite its simplicity, this method is sufficient to win 48 out of 50 games against the Random Player algorithm.

2.3 Kozak's algorithm

Let us assume that the opponent's colour is $\bar{c}$ and ours is c . If after clobbering c 's stone, it is possible to clobber the one ($\bar{c}$'s stone) that was moved by another c 's stone, then we will say that the player of colour c can 'counter' (in the local context). Thus to take one example, in the position , White can 'counter' the only Black move. Or to take another example, in the position , Black can not counter either of White's moves. A simple strategy, which uses two techniques of the choice of a player's move, has been applied to the algorithm. The first 25 player's moves are made according to a 'neighbourhood' rule, while all the subsequent moves are determined on the basis of a 'counter-attack' rule.

The 'neighbourhood' rule relies on the summation of $\bar{c}$'s (the opponent's) stones adjacent to c 's stone s , assuming that s occupies a new square as if the move were already made. All possibilities, resulting from every possible movement, are taken into consideration. As a result, a move that will give the opponent as few options as possible, but more than 0, is chosen. Take, as an illustration, position (i) in Figure 3. There are five moves for White. The



Figure 3: Two exemplary Clobber positions for the 'neighbourhood' rule.

two in the bottom left corner leave no options for Black. The one on the upper side leaves two options for Black. Of the remaining two moves, the first—move up—leaves two options for Black, whereas the second—move down—leaves only one option for Black. Therefore the last one would have been chosen if it was White to play. If all c 's moves leave no options for $\bar{c}$, the opposite turn (a player $\bar{c}$ against a player c) is checked. In this case, if, after clobbering c 's stone s by $\bar{c}$'s stone t , the player of colour c can not counter, then the move ' $s \rightarrow t$ ' is preferred. Take, as an illustration, position (ii) in Figure 3. There are six moves for White, but the one with the rightmost White stone is preferred, because for the opposite move (the rightmost Black stone moving up) White can not counter.

The 'counter-attack' rule relies on checking all $\bar{c}$'s moves with respect to the capability of c to counter. If clobbering c 's stone s by any $\bar{c}$'s stone after which c is not able to counter is possible, then an attempt to block this opportunity is made by moving s in either direction.

Finally, when there is more than one candidate for a move or there are no candidates due to an active rule, a move is selected randomly; in the former case, from the candidates, in the latter case, from among all the feasible moves.

2.4 Scorpu2's algorithm

This algorithm is two-phased and relies on a simple, well-known Clobber heuristic: form large groups of one's stones and prevent the opponent from doing so. In the first phase, from those moves shown in Figure 4, as many ones are made as possible, regardless of the opponent's answers. The moves towards the side of a board

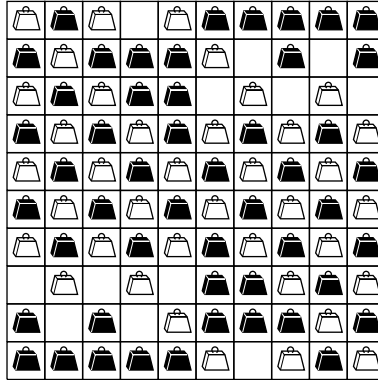


Figure 4: The opening moves.

are chosen earlier, those towards the center, later. Such an arrangement of stones might turn into a substantial advantage in the endgame. The second phase of the algorithm is based on the evaluation of a position, and is presented as Procedure FindMove2.

Procedure FindMove2($\mathcal{B}$, c)

```

1 for each stone  $s$  on a board  $\mathcal{B}$  do
2   if the colour of  $s$  is  $c$  then
3      $v(s) \leftarrow$  the number of adjacent  $\bar{c}$ 's stones
4   else
5      $v(s) \leftarrow 0$ 
6  $s^* \leftarrow$  a randomly selected stone from  $\operatorname{argmax}_{s \in \mathcal{B}} v(s)$ 
7 return a move with  $s^*$  in a randomly selected direction

```

Our approach is simple, logical, and computationally inexpensive. As for shortcomings, the groups of stones which are formed in the first phase of the algorithm can be easily destroyed (may become broken) or excluded (may become isolated).

3. CONCLUSIONS

In this paper, we constructed some algorithms for playing the game of Clobber. We therefore faced the following tasks: give heuristics based on the analysis of groups of stones, patterns, relations between easy to count parameters, etc., which will be helpful in the beginning and middlegame; and find position estimation functions which will be serviceable in the middlegame and endgame. We proposed a few solutions to these tasks, but it seems that Clobber can still be considered to be far from unravelled. In additional experiments, our best program (Head-scratcher) did not obtain a perfect score against all the remaining described programs (61:39 vs Rafal, 97:3 vs Kozak, and 98:2 vs Scorpu2). In the end it is hard to escape the obvious conjecture (Grossman, 2004; Willemson, 2005): little is known regarding what constitutes 'good' play in the beginning of a game, and even the best currently known heuristics fail quite often.

ACKNOWLEDGEMENTS

We thank the following computing centres where the preliminary computations of our algorithms were carried out: The Academic Computer Centre in Gdańsk TASK, The Academic Computer Centre CYFRONET AGH, Kraków (computing grant 027/2004), The Wrocław Centre for Networking and Supercomputing (computing grant 04/97), The Interdisciplinary Centre for Mathematical and Computational Modelling, Warsaw University (computing grant G27-9), and The Poznań Supercomputing and Networking Centre.

4. REFERENCES

- Albert, M., Nowakowski, R., and Wolfe, D. (2007). *Lessons in play: an introduction to combinatorial game theory*. AK Peters. ISBN 978-1-56881-277-9.
- Albert, M. H., Grossman, J. P., Nowakowski, R. J., and Wolfe, D. (2005). An introduction to Clobber. *INTERS: The Electronic Journal of Combinatorial Number Theory*, Vol. 5, No. 2.
- Berlekamp, E. R., Conway, J. H., and Guy, R. K. (2003). *Winning Ways for Your Mathematical Plays*, Vol. 3. AK Peters. ISBN 1-56881-143-8.
- Conway, J. H. (2000). *On Numbers and Games*. AK Peters, Ltd., 2nd edition. ISBN 1-56881-127-6.
- Demaine, E. D., Demaine, M. L., and Fleischer, R. (2004). Solitaire Clobber. *Theoretical Computer Science*, Vol. 313, No. 3, pp. 325-338.
- Grossman, J. (2004). Report on the First International Clobber Tournament. *Theoretical Computer Science*, Vol. 313, No. 3, pp. 533-537.
- Hearn, R. A. and Demaine, E. D. (2009). *Games, Puzzles, and Computation*. A K Peters. ISBN 978-1-56881-322-6.
- Russell, S. and Norvig, P. (2002). *Artificial intelligence: a modern approach*. Prentice Hall. ISBN 0-13-790395-2.
- Siegel, A. (2007). Combinatorial Game Suite. <http://www.cgsuite.org/>.
- Wieczorek, W., Skinderowicz, R., Kozak, J., and Juszczuk, P. (2011). Programs and server for 2011 Clobber tournament. <http://ii.us.edu.pl/clobber/>.
- Willemson, J. (2005). A Computer-Clobber Tournament at the Tartu University. *ICGA Journal*, Vol. 28, No. 1, pp. 51-54.
- Willemson, J. (2008). Rules and protocol for 2006 Clobber tournament. <http://ticc.uvt.nl/icga/games/Clobber/>.

5. APPENDICES

APPENDIX A: THE TOURNAMENT RESULTS

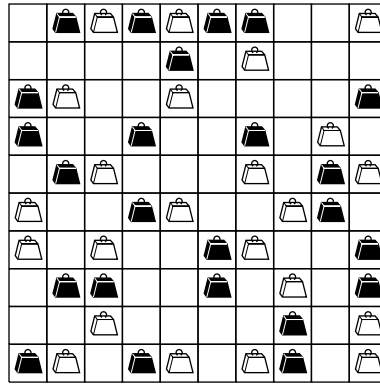
The First Silesian University Computer Clobber Tournament took place at the Institute of Computer Science in Sosnowiec, Poland, on June 22, 2011. Nine players registered for the tournament; their names, the names of their programs, and their results are given in Table 1. As can be seen from the table, it was an all-play-all tournament, where in a single schedule, each participant played every other participant five times. The participants of the tournament agreed upon the following conditions: the board size, ten by ten, the time for a single game, 180 seconds per side, and the games to be played automatically using a specially defined IP-based protocol on an Intel Core i5-2500, 3.2 GHz processor under the Windows 7 operating system with 4 GB RAM. The tournament rules and protocol specifications can be found on the web-page (Willemson, 2008). The programs and MGameServer (ver. 0.4.1 on the GNU General Public License) can be downloaded from another web-page, (Wieczorek *et al.*, 2011).

Table 1: Results of the tournament.

Place	Names	WW	RS	JK	PJ	RK	PS	AK	TU	KD	Sum
I	W. Wieczorek (Head-scratcher)		4:1	5:0	5:0	5:0	4:1	5:0	5:0	5:0	8
II	R. Skinderowicz (Rafal)	1:4		4:1	5:0	4:1	5:0	5:0	4:1	5:0	7
III	J. Kozak (Kozak)	0:5	1:4		4:1	5:0	3:2	5:0	3:2	5:0	6
IV	P. Juszczuk (Scorpu2)	0:5	0:5	1:4		3:2	4:1	4:1	2:3	5:0	4
IV	R. Kraśnicki (Kras1)	0:5	1:4	0:5	2:3		3:2	5:0	5:0	5:0	4
VI	P. Smagała (Deadeye)	1:4	0:5	2:3	1:4	2:3		5:0	3:2	5:0	3
VII	A. Kesy (Beavis)	0:5	0:5	0:5	1:4	0:5	0:5		5:0	5:0	2
VII	T. Urbański (Urba1)	0:5	1:4	2:3	3:2	0:5	2:3	0:5		5:0	2
IX	K. Dworak (Revelation)	0:5	0:5	0:5	0:5	0:5	0:5	0:5	0:5		0

APPENDIX B: A SELECTED GAME

The position in the diagram (see Figure 5) arose in Head-scratcher vs Rafal (Sosnowiec) 2011. The game had begun this way: 1 i1xh1 a2xb2 2 a3xb3 c8xb8 3 c9xb9 b5xb6 4 c7xb7 a10xb10 5 b7xb6 j7xi7 6 h8xh7 f9xe9 7 h7xi7 i2xj2 8 c1xb1 e4xe3 9 d8xd9 a6xa5 10 d9xe9 f7xf6 11 b9xb10 e6xe7 12 f2xe2 b2xb1 13 e2xe3 d3xd4 14 d6xd7 f5xe5 15 g5xg6 g8xg9 16 b4xc4 i8xi7 17 a9xa8 h9xh10 18 h6xi6 i4xh4 19 c5xd5 d4xc4 20 f8xe8 f1xe1 21 g3xh3 h4xh3 22 i3xj3 j5xj6 23 i9xi10 j9xj10 24 d2xd1 g2xg1 25 i10xh10 f6xg6 26 h10xg10 e7xe8 27 e3xf3.

**Figure 5:** The position after 27 e3xf3.

We will consider the game from the perspective of Head-scratcher's algorithm. The last move was the first to be the product of the execution of lines 16–19 in Procedure FindMove. Let us show how the position was evaluated. The value of the largest block, which contains the b10 stone, equals 1. This means that Black can force a win in this local area regardless of who moves first; in addition, there is one move which, if played by White, arranges that the local area will no longer be in the class $\mathcal{L}$. The value of the block that contains the a8 stone also equals 1. The same for f4's block. The block that contains the g7 stone is a member of the class $\mathcal{N}$. Its value equals $(1, -1)$. There are four other such two-stone blocks. The block that contains the i7 stone is in the class $\mathcal{P}$ (the one who plays first, loses). Naturally, its value equals 0. The other instances of the class $\mathcal{P}$ are: the block that contains the j4 stone and the block that contains the h3 stone. Finally, the value of the block that contains the c4 stone equals $(1, -2)$. This means that Black can get the value 0 in one move, and White can get the value -1 in one move. Now let us sum up:

$$\text{Evaluate}(\mathcal{B}, \text{White}) = (1, -2) + 5 \times (1, -1) + 3 \times 1 = -2 + 1 - 1 + 1 - 1 + 1 + 3 = 2.$$

Hence the game is promising for Black. Indeed, the value of the position can be computed to be equal to $\{6 \cdot \uparrow * \mid \uparrow * \parallel \uparrow * \}$ (which belongs to class $\mathcal{L}$) with the help of Combinatorial Game Suite (Siegel, 2007). Unfortunately, not always do $\text{Evaluate}(b, c) > 0 \wedge b \in \mathcal{L}$ or $\text{Evaluate}(b, c) < 0 \wedge b \in \mathcal{R}$ hold for a position b and colour c .

27... g4xf4 28 c3xc2 b8xa8 29 d5xe5 g9xg10 30 d10xc10 j2xj3 31 a7xa8 h5xi5 32 g7xg6 b1xa1 33 i6xi5 e1xd1 34 j4xj3 g1xh1 and Black finally forced a win using the ability to read all paths to the end (line 2 in Procedure FindMove) 35 b6xc6 h1xh2 36 f10xg10 f4xf3 37 e9xe8.